Migración de una Aplicación de Realidad Virtual de la Universidad Politécnica de Victoria de Java a Kotlin para Android

Nicolas Animas-Arguello *, Jesus Emmanuel Lopez-Zuñiga *,
Myriam Elizabeth Pequeño-Aguilera*, Enrique Ramos-Gallegos*
*Ingeniería en Tecnologías de la Información
Universidad Politécnica de Victoria

***Resumen*—Este trabajo presenta la migración de una aplicación de realidad virtual para Android, desarrollada originalmente en Java, hacia Kotlin. La aplicación permite recorrer virtualmente las instalaciones de la Universidad Politécnica de Victoria mediante un entorno inmersivo y un control de realidad virtual. Durante la migración se convirtieron las clases del proyecto, se adaptó la configuración necesaria y se verificó el correcto funcionamiento de la aplicación. Como resultado, se obtuvo un proyecto funcional en Kotlin con una estructura de código más moderna y fácil de mantener.**

I. Introducción

La realidad virtual (RV) ha experimentado un crecimiento significativo en la última década, impulsada por el desarrollo de dispositivos de visualización, sensores de movimiento y plataformas de desarrollo que permiten crear entornos tridimensionales inmersivos a un costo cada vez más accesible. Esta tecnología facilita a los usuarios la sensación de presencia dentro de un entorno digital mediante la interacción directa con objetos y escenarios, superando las limitaciones de las interfaces tradicionales. Gracias a estas capacidades, la RV se ha integrado exitosamente en ámbitos diversos, tales como la educación, la medicina, la arquitectura, la ingeniería, la capacitación industrial y el entretenimiento [1, 2].

En el ámbito educativo, la RV se ha consolidado como una herramienta potente para mejorar el proceso de enseñanza-aprendizaje, permitiendo simular entornos o situaciones de difícil acceso en un aula convencional. Investigaciones han demostrado que estos entornos incrementan la motivación del estudiante y fomentan el aprendizaje activo, mejorando notablemente la comprensión espacial en actividades de exploración técnica [1, 3]. Además, la implementación de recorridos virtuales ofrece una solución estratégica para que estudiantes de nuevo ingreso y visitantes conozcan la infraestructura universitaria de forma remota, superando barreras geográficas.

Este desarrollo tecnológico ha sido respaldado por la evolución del sistema operativo Android, el cual se mantiene como la plataforma móvil con mayor adopción a nivel global. Android proporciona un robusto ecosistema de herramientas, bibliotecas y APIs que permiten implementar aplicaciones con capacidades gráficas avanzadas y soporte nativo para tecnologías de RV. Por ello, gran parte de las soluciones inmersivas actuales utilizan motores gráficos especializados sobre esta plataforma para el procesamiento de entornos tridimensionales y la gestión de eventos de interacción [4, 5].

Durante años, Java fue el lenguaje predominante para el desarrollo en Android debido a su madurez y estabilidad. No obstante, ante el aumento en la complejidad de las aplicaciones, surgieron desafíos técnicos relacionados con el manejo de referencias nulas (NullPointerExceptions), la verbosidad del código y los costos de mantenimiento a largo plazo. Estas limitaciones motivaron la transición hacia alternativas más modernas.

Como respuesta, surgió Kotlin, un lenguaje desarrollado por JetBrains que se ejecuta sobre la Máquina Virtual de Java (JVM), garantizando interoperabilidad total con el código existente. En 2017, Google otorgó a Kotlin soporte oficial como lenguaje de primera clase para Android, recomendándolo posteriormente como el estándar para el desarrollo de nuevas aplicaciones [6, 7]. Kotlin destaca por características avanzadas como la seguridad contra referencias nulas (Null Safety), inferencia de tipos, corrutinas para programación asíncrona y una sintaxis concisa que reduce drásticamente el código repetitivo (boilerplate), optimizando así la legibilidad y la productividad del desarrollador [8]. Asimismo, la compatibilidad total con Java permite realizar migraciones incrementales, reduciendo riesgos en la modernización del software.

La literatura científica respalda esta transición: Hecht et al. [9] analizaron la adopción de Kotlin en proyectos de código abierto, reportando una tendencia creciente tras su oficialización. Por otro lado, Pinto et al. [10] concluyen que Kotlin mejora directamente la mantenibilidad y reduce la incidencia de errores comunes presentes en implementaciones originales en Java, lo cual valida la migración como una estrategia eficaz de mejora de calidad.

En este contexto, el presente trabajo aborda la migración de una aplicación de realidad virtual, desarrollada originalmente para la Universidad Politécnica de Victoria, hacia el lenguaje Kotlin. La aplicación permite realizar un recorrido interactivo por el campus mediante el uso de visores de RV. El proyecto constituye un caso de estudio idóneo para ejecutar un proceso de migración que comprende la conversión de clases, la adaptación de la configuración del proyecto y la validación de la estabilidad del sistema tras el cambio de paradigma.

El objetivo central es realizar esta transición preservando la funcionalidad original y aprovechando las ventajas de Kotlin en términos de seguridad, robustez y mantenibilidad, asegurando que la experiencia del usuario final en el recorrido virtual se mantenga intacta tras la actualización tecnológica.

II. Desarrollo Experimental

El proceso consistió en la migración completa de una aplicación Android implementada originalmente en Java hacia Kotlin, procurando conservar la funcionalidad, la organización del proyecto y la arquitectura definida en la versión original. Debido a que la aplicación representa un sistema con múltiples componentes interdependientes, la migración se realizó de forma incremental, analizando previamente la estructura del proyecto y estableciendo una metodología que permitiera reducir errores de compilación y facilitar la identificación de incompatibilidades entre ambos lenguajes.

Antes de iniciar la conversión fue necesario estudiar la organización interna de la aplicación con el propósito de identificar las relaciones entre actividades, clases auxiliares, componentes gráficos y recursos del proyecto. A partir de este análisis se definió una estrategia de migración orientada a preservar la estructura del software y garantizar que cada modificación pudiera validarse antes de continuar con el siguiente componente.

II-A. Análisis de la aplicación original

Como etapa inicial se realizó un análisis de la estructura del proyecto con el objetivo de identificar los componentes que conforman la aplicación y determinar las dependencias existentes entre ellos. Este análisis permitió conocer la organización de los paquetes, la distribución de las clases y la interacción entre las actividades principales, los modelos de datos, las utilerías y los componentes encargados del renderizado gráfico.

La identificación de esta arquitectura permitió establecer una estrategia de migración que respetara la organización original del proyecto, conservando la estructura de paquetes, la nomenclatura de las clases y las dependencias existentes entre los diferentes módulos.

Asimismo, se verificó que el proyecto compilara correctamente en su versión desarrollada en Java, estableciendo una línea base de funcionamiento que posteriormente serviría como referencia para validar la versión migrada.

II-B. Arquitectura de la aplicación

La Figura 1 presenta la arquitectura general de la aplicación original, organizada en cuatro componentes principales: la capa de presentación, la lógica de la aplicación, el motor de renderizado y los recursos del proyecto. Las actividades (*Activities*) constituyen el punto de interacción con el usuario y son responsables de gestionar los eventos y coordinar la ejecución de la lógica del sistema. Esta última procesa la información, administra los modelos de datos y controla el flujo de ejecución de la aplicación.

Por su parte, el motor de renderizado implementa las operaciones necesarias para la representación de los objetos tridimensionales mediante OpenGL ES, haciendo uso de modelos, texturas y demás recursos gráficos almacenados en el proyecto. Finalmente, los recursos de Android, como archivos XML, imágenes y configuraciones, son compartidos entre los distintos componentes para proporcionar tanto la interfaz gráfica como los elementos necesarios para el funcionamiento del sistema.

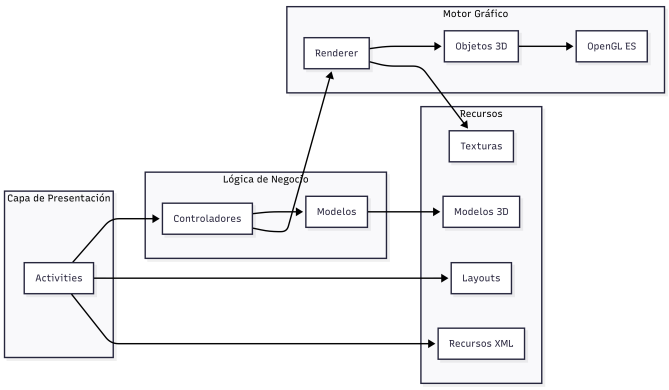


Figura 1: Diagrama de componentes que muestra la organización funcional de la aplicación antes del proceso de migración a Kotlin.

II-C. Planeación de la migración

Una vez comprendida la arquitectura del proyecto se definió una estrategia de migración incremental. El objetivo principal consistió en sustituir gradualmente los archivos implementados en Java por sus equivalentes en Kotlin sin alterar la estructura del proyecto ni modificar el comportamiento observable de la aplicación.

Como criterio de trabajo se decidió conservar la organización original de paquetes, la nomenclatura de las clases, la distribución de recursos y el número de archivos existentes en el proyecto. De esta manera fue posible

mantener la compatibilidad entre los distintos componentes y facilitar la comparación entre ambas versiones durante el proceso de validación.

II-D. Metodología de la migración

Con base en el análisis de la arquitectura del proyecto, se definió una metodología de migración incremental orientada a minimizar la introducción de errores y facilitar la validación de cada cambio realizado. La estrategia consistió en migrar un único archivo a la vez, verificando su correcta integración con el resto del sistema antes de continuar con el siguiente componente. La Figura 2 resume el flujo de trabajo seguido durante el proceso de migración.

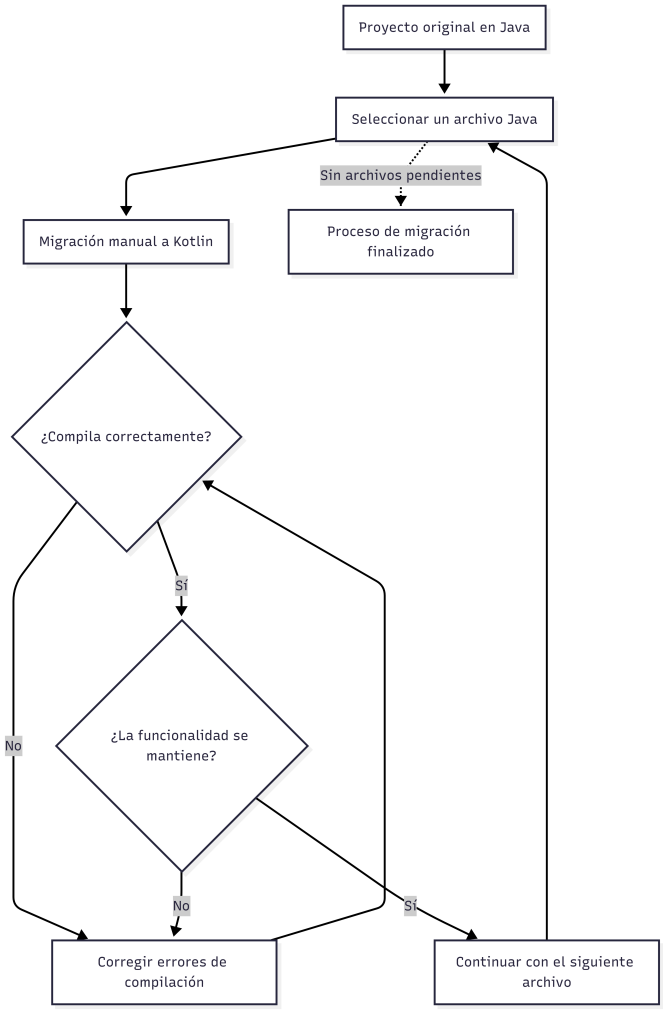


Figura 2: Flujo de trabajo seguido durante el proceso de migración, compilación y validación de cada archivo del proyecto.

II-E. Modificaciones realizadas

Al finalizar la migración, se identificó y dio solución a dos fallos principales:

II-E1. Corrección de movimiento opuesto en los controles:

- ***Fallo detectado:*** En la versión Java, algunos vectores de desplazamiento y rotación respondían de manera invertida debido a incongruencias en la inversión de signos dentro de la lógica matemática de navegación.
- ***Solución Aplicada:*** En la nueva versión en Kotlin, se ajustaron los factores de multiplicación vectorial, eliminando los operadores negativos innecesarios, lo que permitió que las direcciones del control correspondieran de manera natural e intuitiva con el desplazamiento del usuario.

II-E2. Corrección de desplazamiento diagonal no deseado al avanzar o retroceder:

- ***Fallo detectado:*** Al desplazarse hacia adelante o hacia atrás, el sistema en Java utilizaba el vector de dirección completo. Dado que este vector se recalcula directamente a partir de la matriz de rotación del dispositivo para incluir la inclinación vertical (mirar hacia arriba o hacia abajo), el usuario experimentaba una alteración involuntaria de la altura, generando un movimiento diagonal no deseado (flotando o hundiéndose ligeramente).
- ***Solución Aplicada:*** En la nueva versión en Kotlin, se modificó la función *MoveForward* aislando únicamente la componente horizontal del vector de dirección. Al ignorar el eje Y para este cálculo específico, se logró que el movimiento de avance y retroceso se mantuviera estrictamente plano sobre el suelo, preservando el vector completo exclusivamente para la renderización de la cámara.

III. Resultados

Una vez concluido el proceso de migración se obtuvo una versión funcional de la aplicación desarrollada completamente en Kotlin, manteniendo la arquitectura, la organización del proyecto y el comportamiento general de la versión implementada originalmente en Java. Los resultados obtenidos permiten evaluar el grado de éxito alcanzado durante la conversión y verificar el cumplimiento de los objetivos planteados.

III-A. Resultado de la migración

La Tabla I resume los principales resultados obtenidos durante la migración. Se observa que el número de archivos se mantuvo constante, lo que permitió conservar la organización original del proyecto y facilitar la comparación entre ambas versiones de la aplicación.

Tabla I: Resumen del proceso de migración de la aplicación.

Métrica	Resultado
Lenguaje original	Java
Lenguaje final	Kotlin
Archivos Java originales	61
Archivos Kotlin generados	69
Estructura de paquetes	Conservada
Nomenclatura de clases	Conservada
Recursos del proyecto	Sin modificaciones
Arquitectura de la aplicación	Conservada
Método de migración	Manual e incremental

III-B. Conservación de la arquitectura

Uno de los principales objetivos de la migración consistió en preservar la arquitectura de la aplicación. Para ello se mantuvo la estructura de paquetes, la organización de las clases y las relaciones de dependencia entre los distintos componentes del sistema. Asimismo, los recursos gráficos, archivos de configuración y demás elementos del proyecto permanecieron sin modificaciones, limitando los cambios únicamente al lenguaje de programación utilizado en la implementación.

La conservación de esta estructura permitió reducir el impacto de la migración sobre el funcionamiento general de la aplicación y simplificó la validación de los componentes migrados.

III-C. Verificación funcional

Con el propósito de validar la correcta operación de la aplicación después del proceso de migración, se realizaron pruebas de ejecución sobre los principales escenarios de uso. Estas pruebas consistieron en verificar la carga de los modelos tridimensionales, la navegación dentro del entorno virtual y el acceso al interior de los edificios disponibles en la aplicación. La Figura 3 presenta algunas de las capturas obtenidas durante estas pruebas funcionales.

Las capturas muestran que la aplicación conserva la capacidad de representar correctamente los modelos tridimensionales y de cargar los recursos gráficos asociados a cada edificio. Asimismo, la visualización del interior del edificio I confirma que la navegación dentro del entorno virtual continúa funcionando de manera adecuada, lo que indica que la migración de Java a Kotlin no afectó la operación del motor de renderizado ni la interacción con la escena tridimensional.

Además, con la finalidad de probar si los errores de la aplicación original en Java fueron realmente solucionados en la nueva versión Kotlin, se realizaron pruebas relacionadas con el control de movimiento y la navegación del usuario en el entorno.

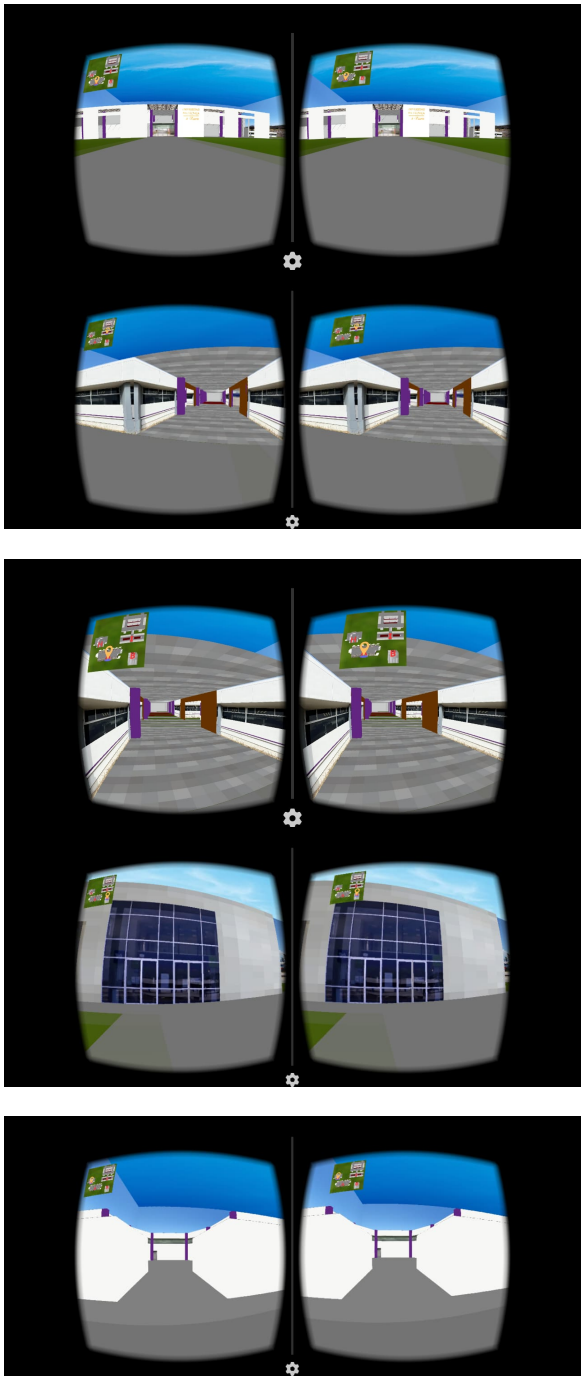


Figura 3: Capturas obtenidas durante la ejecución de la aplicación migrada a Kotlin. Se observa la visualización de los edificios A, I, B y H, así como una vista del interior del edificio I, evidenciando el correcto funcionamiento del motor de renderizado y la carga de los recursos gráficos.

La evidencia en video puede ser observada en el siguiente repositorio:

https://github.com/EmmanuelLopez15/video_movimiento_upv

IV. Conclusión

La migración de la aplicación de realidad virtual de la Universidad Politécnica de Victoria desde Java hacia Kotlin permitió modernizar la base de código sin afectar las funcionalidades principales del sistema. Durante el proceso fue posible convertir las clases existentes, adaptar la configuración del proyecto y resolver los problemas de compatibilidad propios de la interoperabilidad entre ambos lenguajes, verificando posteriormente el correcto funcionamiento de la aplicación.

Los resultados obtenidos demuestran que Kotlin representa una alternativa adecuada para la evolución de aplicaciones Android desarrolladas originalmente en Java, ya que ofrece una sintaxis más concisa, un mejor manejo de referencias nulas y herramientas que facilitan el mantenimiento y la escalabilidad del software. Asimismo, la compatibilidad entre ambos lenguajes permitió realizar una migración gradual, reduciendo el riesgo de introducir errores y evitando la necesidad de reescribir completamente el proyecto.

Como resultado de este trabajo se obtuvo una aplicación funcional que conserva la experiencia de recorrido virtual por las instalaciones de la Universidad Politécnica de Victoria, permitiendo al usuario explorar el entorno mediante un control de realidad virtual tal como en la versión original. Además, el proyecto quedó preparado para incorporar futuras mejoras, actualizar dependencias y aprovechar nuevas características del ecosistema Android.

Como trabajo futuro, se plantea optimizar el rendimiento de la app e incorporar nuevas funcionalidades que mejoren la interacción del usuario, como sistemas de información contextual, recorridos guiados y una interfaz más intuitiva, fortaleciendo así la experiencia ofrecida por la aplicación.

Referencias

[1] Jaziar Radianti et al. “A Systematic Review of Immersive Virtual Reality Applications for Higher Education: Design Elements, Lessons Learned, and Research Agenda”. En: *Education and Information Technologies* 25.6 (2020), págs. 5421-5452. DOI: 10.1007/s10639-019-10063-7.

[2] Zahira Merchant et al. “Effectiveness of Virtual Reality-Based Instruction on Students’ Learning Outcomes in K–12 and Higher Education: A Meta-analysis”. En: *Computers & Education* 70 (2014), págs. 29-40. DOI: 10.1016/j.compedu.2013.07.033.

[3] Laura Freina y Michela Ott. “A Literature Review on Immersive Virtual Reality in Education: State of the Art and Perspectives”. En: *Proceedings of the International Scientific Conference eLearning and Software for Education*. Vol. 1. 2015, págs. 133-141. DOI: 10.12753/2066-026X-15-020.

[4] Google. *Android Developers*. Consultado en junio de 2026. 2024. URL: https://developer.android.com.

[5] Google. *ARCore Overview*. Consultado en junio de 2026. 2024. URL: https://developers.google.com/ar.

[6] Android Developers. *Kotlin for Android*. Consultado en junio de 2026. 2024. URL: https://developer.android.com/kotlin.

[7] JetBrains. *Kotlin Programming Language*. Consultado en junio de 2026. 2024. URL: https://kotlinlang.org.

[8] JetBrains. *Kotlin Documentation*. Consultado en junio de 2026. 2024. URL: https://kotlinlang.org/docs/home.html.

[9] Geoffrey Hecht, Ivano Malavolta y Others. “Understanding the Adoption of Kotlin in Android Development”. En: *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019.

[10] Gustavo Pinto et al. “An Empirical Study of the Usage of Kotlin in Android Applications”. En: *Journal of Systems and Software* 182 (2021), pág. 111067. DOI: 10.1016/j.jss.2021.111067.